

**Fast National University of computing and Emerging Sciences
Peshawar Campus**



Data Structures

Practice Problem(Binary Tree)

Basic Problems:

1. Create a Binary Tree:

Implement a binary tree with basic insertion and traversal methods (inorder, preorder, postorder).

2. Find Height of a Binary Tree:

Write a function to compute the height of a binary tree.

3. Count Nodes in Binary Tree:

Write a function to count the number of nodes in a binary tree.

4. Find Maximum Element in a Binary Tree:

Write a function that returns the maximum value stored in a binary tree.

5. Find Minimum Element in a Binary Tree:

Write a function that returns the minimum value stored in a binary tree.

6. Level Order Traversal:

Implement level-order traversal (BFS) of a binary tree.

7. Depth-First Traversals:

Implement preorder, inorder, and postorder traversals of a binary tree.

8. Check if Two Trees are Identical:

Write a function to check if two binary trees are identical.

9. Sum of All Nodes:

Write a function to return the sum of all nodes in a binary tree.

10. Count Leaf Nodes:

Write a function to count all the leaf nodes in a binary tree.

Intermediate Problems:

11. Check if a Binary Tree is Balanced:

Write a function to check if a binary tree is height-balanced.

12. Diameter of a Binary Tree:

Write a function to find the diameter (longest path between any two nodes) of a binary tree.

13. Convert a Binary Tree to its Mirror:

Implement a function that converts a binary tree into its mirror image.

14. Check if a Binary Tree is a Sum Tree:

Write a function to check if a binary tree is a sum tree (where each node is equal to the sum of its left and right children).

15. Check for Symmetry in Binary Tree:

Write a function to check if a binary tree is symmetric.

16. Find Lowest Common Ancestor (LCA):

Implement a function to find the lowest common ancestor of two nodes in a binary tree.

17. Maximum Path Sum:

Write a function to find the maximum sum path between any two leaves in the binary tree.

18. Check if a Binary Tree is a Binary Search Tree (BST):

Implement a function to check if a binary tree is a valid binary search tree (BST).

19. Binary Tree Paths:

Write a function that prints all paths from root to leaf nodes in a binary tree.

20. Serialize and Deserialize Binary Tree:

Write functions to serialize a binary tree to a file and deserialize it back.

Advanced Problems:

21. Inorder Successor in Binary Search Tree:

Implement a function to find the inorder successor of a node in a binary search tree.

22. Build Binary Tree from Preorder and Inorder Traversals:

Given the preorder and inorder traversal arrays, construct the binary tree.

23. Build Binary Tree from Postorder and Inorder Traversals:

Given the postorder and inorder traversal arrays, construct the binary tree.

24. Zigzag Level Order Traversal:

Implement a zigzag (spiral) level-order traversal of a binary tree.

25. Flatten a Binary Tree to a Linked List:

Write a function to flatten the binary tree into a linked list in-place.

26. Check if a Binary Tree has a Root-to-Leaf Path with Given Sum:

Write a function to check if there exists a root-to-leaf path with a given sum.

27. Right View of Binary Tree:

Write a function to print the right view of a binary tree.

28. Left View of Binary Tree:

Write a function to print the left view of a binary tree.

29. Boundary Traversal of Binary Tree:

Implement a function to perform boundary traversal of a binary tree.

30. Count Subtrees with Given Sum:

Write a function to count all subtrees in a binary tree that have a given sum.